

Final Sprint – Fall 2025

Gym Management System

Project Overview

You have been tasked with developing a **console-based Gym Management System**. This project will simulate the management of a gym, allowing different users—**Admins, Trainers, and Members**—to interact with the system based on their roles.

The project should be implemented **as a team effort**, with each group consisting of up to three students. **Every team member should actively contribute to the project.**

Solos will be considered on a case-by-case basis. Please reach out if you have not already done so!

Make sure to mark your name on the sheet so we can keep track of who is in what group 😊

Team/Solo Sign Up: [S3 Sprint Teams Fall 2025.xlsx](#)

Project Objectives

- Develop a **Java application** that manages gym users, memberships, and workout classes.
- Implement **user authentication** and **role-based access control (RBAC)** for Admins, Trainers, and Members.
- Integrate the system with a **PostgreSQL database** to store and manage user, membership, and workout class data.
- Demonstrate **effective teamwork and project management skills** using **GitHub and other tools**.

Functional Requirements

User Registration and Authentication

- Users can register with a **username, password, email, phonenumber, address and role** (Admin, Trainer, or Member).
- Registered users can log in and see a **role-specific menu**.
- Passwords must be **securely stored** using **BCrypt hashing**. **Passwords can be hashed before being saved to the Database** (Maven dependency or adding JAR file manually required in order to access BCrypt).
- Example classes:
 - User, UserDAO, UserService
 - Admin, Trainer, Member (Inheritance from User)

Product Management: Memberships & Classes

Membership Management

- Members and trainers can purchase a **gym membership**.
- Admins can view and track **total revenue from memberships in the gym**.
- Example classes: Membership, MembershipDAO, MembershipService

Workout Class Management

- Trainers can **add, update, and delete** workout classes.
- Members can **view a list of workout classes in the system**
- Example classes: WorkoutClass, WorkoutClassDAO, WorkoutClassService

Gym Merchandise Management (Workout gear, drinks, food)

- Members can view all the products available at the gym to purchase
- Admins can add new items and set prices, print a report of all items in stock
- Trainers can view all the products like the Member

Role-Based Functionality

Admin

- View all users in the system, including their contact information.
- Delete users from the system.
- View **all gym memberships** and total **annual revenue**.
- Admins can add new items and set prices, print a report of all items in stock, and get the total value of their stock of merch.

Trainer

- Create, update, and delete **workout classes**.
- View a **list of all their assigned classes**.
- Purchase a **gym membership** for themselves.
- Trainers can view all the products like the Member

Member

- Browse workout **classes**.
- View their **total membership expenses**.
- Purchase a **new gym membership**.
- Members can view all the products available at the gym to purchase

Database Integration

- Use **PostgreSQL** to store and manage user, membership, and class information.
- Will need to connect the PGDriver manually (approach shown in class or handle the dependency another way (i.e Maven)
- Implement **CRUD operations** (Create, Read, Update, Delete) for users, memberships, and workout classes.
- Recommended **database schema** can include
 - Users table
(userId,userName,userAddress,userPhoneNumber,userRole...)

- Memberships table
(membershipID, membershipType, membershipDescription, memberShipCost, memberID)
- WorkoutClasses table
(workoutClassID, workoutClassType, workoutClassDescription, trainerID (a trainer is also a user just a different role)
- GymMerch table
(merchid, merchName, merchType, merchPrice, quantityInStock)

User Interface

- Develop a **console-based** user interface for interaction using a scanner.
- Provide **clear menu options** and **navigation prompts** for users.

Logging

- You must implement a logger for your application, you must use the logger in some way, and it must write to a text file, The approach you take is up to you.

What is logging?

- Logging means recording messages from your program that describe events, data, or errors as they happen. These messages are usually written to the console, a file, or an external logging service.

Technical Requirements

- **Programming Language:** Java
- **Database:** PostgreSQL (If you want to try another feel free but PG is recommended)
- **IDE:** IntelliJ IDEA, Eclipse, VS Code, or any preferred Java IDE
- **Version Control:** Git/GitHub (**Mandatory for submission**)
- **Dependency Management:** Maven or Manually adding Jar files will work.

Submission Guidelines

Code Submission – 60 Marks

- Submit the **complete source code** via GitHub.
- **ZIP FILE SUBMISSIONS WILL NOT BE MARKED.** If you need help with GitHub, contact your TA.

Database Schema – 5 Marks

- Provide **SQL scripts** to create and populate your database.

Documentation – 20 Marks

Prepare a **detailed report** containing the following 3 sections:

1. User Documentation (Assume this is for the user who bought the program. They might not be too tech savvy- Provide lots of detail)

- Overview of the application and how it works and what it does
- Explanation of **all classes and their interactions.**
- **Class diagram** showing relationships between entities.
- Instructions on **how to start and use the system.**

2. Development Documentation

- **Javadoc documentation** for key methods and classes.
- **Project directory structure** explanation.
- **Build process** and dependencies used.
- How to **set up the database** for development.
- How to **clone and run the project** from GitHub.

3. Individual Report

- Submit a short **individual report** detailing:
 - **Your contributions** (PRs, branches worked on, contributions to team).
 - **Challenges faced** during development (Could be anything)

Presentation- 15 Marks

- Prepare a short video (5-10 minutes) demonstrating:
 - **Application functionality** (Show role-based features).
 - **Development process** (Explain how the system works).
 - **Team contributions** (Each member will talk about sections they worked on and talk about challenges faced).

Challenges (Optional, Not Required for Submission)

- Run the application in **another IDE** besides the one you used for development.
- Implement **advanced error handling** using try-catch (as demonstrated in class).
- Show the use of an interface in the code
- Expand on the program in some way

Requirements for Submission

- All classes as outlined above
- Documentation – Must meet all requirements as outlined above
- Video Presentation- All team members must take part in this!
- Project must be submitted using GitHub not a ZIP file. Zips will not be looked at

Important Dates

- **Project Kickoff: November 30th**, 2025
- **Code Submission Deadline: December 17th**, 2025
- ★ *There will be no room for any extensions so please manage time efficiently*

Resources

- [Java Documentation](#)
- [GitHub Guides](#)
- [PostgreSQL Documentation](#)
- [Example repository from TA session](#)
- [Template Repo If Needed!](#)

- To use a template, just click the USE TEMPLATE button on the top right side of the page. Button should be blue

Instructor Support

Your **instructor and TA** are available to provide guidance throughout the project.

- **Reach out for help if you are stuck. Do not get stuck for too long that is not fun at all**

Good Luck and Happy Coding!